Лабораторна робота №11. AI-агент для інтеграції REST API: Fetch API, Promise та async/await, аналіз API-документації, loading та error states, debugging через Network

Термін здачі: 23.10.2026

1. Мета роботи

Навчитися інтегрувати REST API у фронтенд-застосунок за його документацією: надсилати запити через Fetch API, обробляти відповіді й помилки з `async/await`, скасовувати застарілі запити й показувати користувачеві стани завантаження, порожнього результату та помилки. Навчитися перевіряти асинхронний код, написаний AI-агентом, за специфікацією, самоперевіркою й панеллю Network, а не за тим, що «на моєму інтернеті працює».

Очікуваний результат: застосунок ShowFinder на справжньому TVmaze API, кожну частину якого спершу написав AI-агент, а ви знайшли розбіжності з ТЗ і виправили їх власноруч; «Пройдено: 24 з 24» у самоперевірці; конспект API `API.md` і правила для агента `AGENTS.md` у репозиторії.

2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- Надсилати запити через `fetch`, читати `status`, `ok`, заголовки й тіло відповіді; пояснювати, чому `fetch` не відхиляє проміс на 404 і 500.
- Писати асинхронний код з `async/await`: обробляти помилки в `try/catch`, запускати незалежні запити паралельно (`Promise.all`), розрізняти `return` і `return await`.
- Скасовувати застарілі запити через `AbortController` і відрізняти скасування від помилки.

- Пояснювати статуси 200, 301, 404, 429 і 5xx та що кожен означає для інтерфейсу; заголовки `Content-Type` і `Cache-Control`; HTTP-кеш браузера.
- Пояснювати CORS: що саме блокує браузер, коли він надсилає preflight і чому помилку CORS не виправити з боку клієнта.
- Досліджувати запити в панелі Network: статус, заголовки, Payload, Initiator, Timing, throttling, блокування запитів.
- Читати документацію API: ендпоїнти, параметри, форма відповіді, поля, що можуть бути `null`, помилки, кешування й ліміт запитів.
- Перевіряти асинхронний код, написаний AI, і передавати агенту контекст проєкту через файл правил.

3. Завдання лабораторної роботи

1. Опрацювати дослідницькі завдання (§4), провести експерименти й скласти конспект API `API.md` (§4.5).
2. Завантажити starter-проєкт ShowFinder, зафіксувати його в Git без змін і переконатися, що самоперевірка показує «Пройдено: 0 з 24» (§5.1).
3. Виконати Challenges 1–2 (§5.3–5.4), написати `AGENTS.md` (§5.5) і виконати Challenge 3 (§5.6). Для кожного challenge AI-агент пише чернетку, ви фіксуєте її, перевіряєте за ТЗ і виправляєте власноруч.
4. Пройти чек-лист Network (ТЗ, §6) на фінальній версії, опублікувати проєкт на GitHub Pages або Vercel, оформити звіт (§6). Потім — захист (§7).

4. Дослідницькі завдання

Загальна довідка — Using the Fetch API — MDN, HTTP — MDN і Network features reference — Chrome DevTools; API, з яким ви працюєте, описано в документації TVmaze.

Експерименти досліджують поведінку браузера й TVmaze API, а не код застосунку: у них нічого не треба виправляти. Рядок «Де це трапиться» в кожному експерименті показує, у якій частині ShowFinder ця поведінка стане помилкою, якщо її не врахувати.

Сторінка для експериментів. У starter-проєкті (showfinder-starter.zip, той самий, що в §5) є порожня сторінка `experiments.html`. Вона потрібна лише для того, щоб запити мали справжнє походження (origin), з яким браузер перевіряє CORS. Розпакуйте архів, запустіть Live Preview для `experiments.html` і відкрийте її в Chrome або Edge за адресою `http://127.0.0.1:…`, а не у вбудованій вкладці VS Code.

У DevTools (F12, на macOS — Cmd+Option+I):

- **Network:** увімкніть **Preserve log**, виберіть фільтр **Fetch/XHR** і переконайтеся, що **Disable cache** вимкнено (експеримент 8);
- **Console:** сюди вставляйте фрагмент цілим. Console дозволяє `await` без `async`-функції.

Перед кожним експериментом оновлюйте сторінку й очищайте Console і Network. Фрагменти надсилають справжні запити до TVmaze. Не запускайте їх у циклі: TVmaze дозволяє щонайменше 20 запитів за 10 секунд з однієї IP-адреси.

Протокол експерименту:

1. Прочитайте опис, **проаналізуйте** код і запишіть очікуваний результат — відповідь на питання «Проаналізуйте».
2. Оновіть сторінку й вставте фрагмент у Console. Перевірте й те, що показує Network, якщо питання про нього питає.
3. Якщо хоч один очікуваний результат не збігся з фактичним, поясніть, де саме аналіз був хибним.

Питання в рядку «Де це трапиться» — для підготовки до захисту, письмово на них відповідати не треба. Результати зведіть у таблицю — вона входить у звіт:

#	Що досліджено	Мій аналіз	Фактичний результат (Console і Network)	Пояснення (якщо не збіглося)
1	fetch і статус 404	...	...	...

4.1. Fetch API і Promise

Опрацювати:

- Using the Fetch API, Response — MDN
- async function, Using promises — MDN
- Promise.all(), Promise.allSettled() — MDN
- AbortController — MDN

Дослідити: що повертає `fetch` і коли його проміс відхиляється; `ok` і `status`; чому тіло відповіді можна прочитати лише раз; як помилка поводиться в `async`-функції; паралельні й послідовні запити; як скасувати запит і що тоді отримує код.

Експерименти:

1. **fetch і статус 404.** Фрагмент запитує серіал, якого немає.

   ```
   const response = await fetch('https://api.tvmaze.com/shows/999999999');
   console.log(response.ok, response.status);
   console.log(await response.json());
   ```

   Проаналізуйте: чи відхилить `fetch` проміс? Що виведуть обидва `console.log`? Що з'явиться в Console червоним і чи означає це, що код «упав»?
   Де це трапиться: пошук і сторінка серіалу (Challenge 3): відповідь 404 чи 500, яку ніхто не перевірив через `response.ok`, потрапляє в застосунок як дані.
2. **Тіло відповіді читається один раз.** Спершу фрагмент двічі читає тіло однієї відповіді, потім намагається прочитати як JSON HTML-сторінку.

   ```
   const response = await fetch('https://api.tvmaze.com/shows/82');
   console.log((await response.json()).name);
   try {
     await response.json();
   } catch (error) {
     console.log('вдруге:', error.name);
   }
   const page = await fetch('experiments.html');
   try {
     await page.json();
   } catch (error) {
     console.log('HTML:', error.name, error.message);
   }
   ```

   Проаналізуйте: що виведе кожен `console.log`? Чому друге `json()` не повертає ті самі дані? Яку помилку дає HTML і до чого тут символ <?
   Де це трапиться: Challenge 3. Проксі, сторінка входу в Wi-Fi чи сторінка помилки сервера відповідають HTML зі статусом 200. А код, що читає тіло «для логування», ламає справжнє читання.
3. **forEach, for...of і Promise.all.** Три запити акторського складу трьома способами.

`cache: 'no-store'` вимикає HTTP-кеш браузера, щоб кожен запит ішов у мережу.

```
const ids = [82, 139, 33320];
const load = async (id) => (await
fetch(`https://api.tvmaze.com/shows/${id}/cast`, { cache: 'no-store'
})).json();

const counts = [];
ids.forEach(async (id) => counts.push((await load(id)).length));
console.log('forEach:', counts.length);
await new Promise((resolve) => setTimeout(resolve, 2000));
console.log('через 2 с:', counts.length);

let start = performance.now();
for (const id of ids) await load(id);
console.log('for…of:', Math.round(performance.now() - start), 'мс');

start = performance.now();
await Promise.all(ids.map(load));
console.log('Promise.all:', Math.round(performance.now() - start), 'мс');
```

Проаналізуйте: що виведуть `forEach:` і `через 2 с:`? Який з двох останніх способів швидший і приблизно в скільки разів? Як ці запити виглядають у колонці Waterfall панелі Network?
Де це трапиться: сторінка серіалу (Challenge 2). Епізоди й актори, запитані один за одним, — кілька затримок поспіль. А з `forEach` і `async` стан «завантаження» закінчується раніше, ніж приходять дані.

4. **return і return await у try.** Домен `.invalid` гарантовано не існує, тож запит не дійде до сервера.

```
async function load(url) {
  try {
    return fetch(url);
  } catch (error) {
    console.log('catch у load:', error.name);
    return null;
  }
}
const result = await
load('https://api.tvmaze.invalid/shows/82').catch((error) => `поза load:
${error.name}`);
console.log(result);
```

Проаналізуйте: що виведе Console — `catch у load: …` чи `поза load: …`? Чому? Що зміниться, якщо написати `return await fetch(url);`?

Де це трапиться: Challenge 3: `try/catch` навколо `return fetch(…)` нічого не ловить, і збій мережі стає `Uncaught (in promise)`.

5. **Promise.all і Promise.allSettled.** Три запити, один з яких не дійде до сервера.

```
const urls = ['https://api.tvmaze.com/shows/82',
'https://api.tvmaze.invalid/shows/1', 'https://api.tvmaze.com/shows/139'];
try {
  const responses = await Promise.all(urls.map((url) => fetch(url)));
  console.log('all:', responses.map((response) => response.status));
} catch (error) {
  console.log('all:', error.name);
}
const settled = await Promise.allSettled(urls.map((url) => fetch(url)));
console.log('allSettled:', settled.map((result) => result.status));
```

Проаналізуйте: що виведуть `all:` і `allSettled:`? Що сталося з двома вдалими відповідями в першому випадку?
Де це трапиться: сторінка серіалу з паралельними запитами (Challenge 2): якщо один із запитів упав, `Promise.all` відкидає й решту. Коли це правильна поведінка, а коли ні?

6. **AbortController.** Перший запит скасовано до відповіді, другий — після того, як відповідь прийшла, але до читання тіла.

```
const early = new AbortController();
const request = fetch('https://api.tvmaze.com/search/shows?q=girls', {
signal: early.signal });
early.abort();
try {
  await request;
} catch (error) {
  console.log('до відповіді:', error.name);
}
const late = new AbortController();
const response = await fetch('https://api.tvmaze.com/shows/139', { signal:
late.signal });
late.abort();
console.log('після відповіді:', response.status);
try {
  await response.json();
} catch (error) {
  console.log('json():', error.name);
}
```

Проаналізуйте: що виведе кожен рядок? Що покаже колонка Status панелі Network для скасованих запитів?
Де це трапиться: новий пошук скасовує попередній (Challenge 1), вибір іншого серіалу — попередню сторінку (Challenge 2). Скасування — не помилка, і користувач не має бачити «Не вдалося…» (Challenge 3).

4.2. HTTP: статуси, заголовки, кеш

Опрацювати:

- HTTP response status codes: 200, 301, 404, 429, 500, 503 — MDN
- Content-Type, Cache-Control, HTTP caching — MDN
- CORS-safelisted response header — MDN
- URLSearchParams, encodeURIComponent() — MDN

Дослідити: групи статусів і що кожен з них означає для інтерфейсу; що таке ідемпотентний `GET`; які заголовки відповіді бачить скрипт; як `Cache-Control: max-age` керує кешем браузера; як параметри потрапляють у рядок запиту.

Експерименти:

7. **Заголовки відповіді.**

   ```
   const response = await fetch('https://api.tvmaze.com/shows/82');
   console.log(response.headers.get('content-type'));
   console.log(response.headers.get('cache-control'));
   console.log(response.headers.get('date'));
   console.log([...response.headers.keys()]);
   ```

   Проаналізуйте: що виведе кожен рядок? Network показує в Response Headers і `date`, і `server`. Чому скрипт їх не отримує?
   Де це трапиться: відповідь 429 (Challenge 3). Документації багатьох API радять читати `Retry-After`, але для чужого домену скрипт побачить його, лише якщо сервер дозволить це заголовком `Access-Control-Expose-Headers`.
8. **HTTP-кеш.** Три однакові запити. **Disable cache** у Network має бути вимкнено.

   ```
   await fetch('https://api.tvmaze.com/shows/1');
   await fetch('https://api.tvmaze.com/shows/1');
   await fetch('https://api.tvmaze.com/shows/1', { cache: 'no-store' });
   ```

Проаналізуйте: які з трьох запитів підуть у мережу, а які браузер віддасть з кешу? Що покаже колонка Size? Який заголовок з експерименту 7 це визначає? Якщо повторюєте експеримент, перший запит теж прийде з кешу: чому?
Де це трапиться: повторне відкриття серіалу (Challenge 2). Навіщо застосунку власний кеш, якщо браузер уже кешує відповіді?

9. **Переадресація.** `/lookup/shows` знаходить серіал TVmaze за його id в IMDb (документація TVmaze, розділ Show lookup).

```
const response = await fetch('https://api.tvmaze.com/lookup/shows?
imdb=tt0944947');
console.log(response.status, response.redirected, response.url);
console.log((await response.json()).name);
```

Проаналізуйте: що виведуть `status`, `redirected` і `url`? Скільки рядків з'явиться в Network і з якими статусами?
Де це трапиться: розбір статусів у Challenge 3: статусу 301 код застосунку не побачить ніколи, бо `fetch` сам іде за переадресацією.

10. **Рядок запиту.** Фрагмент лише будує дві адреси й нічого не надсилає.

```
const query = 'Law & Order';
const plain = new URL(`https://api.tvmaze.com/search/shows?q=${query}`);
const encoded = new URL(`https://api.tvmaze.com/search/shows?${new
URLSearchParams({ q: query })}`);
console.log(plain.searchParams.get('q'), [...plain.searchParams.keys()]);
console.log(encoded.searchParams.get('q'), encoded.search);
```

Проаналізуйте: яке значення `q` отримає API в кожному випадку? Скільки параметрів у першій адресі?
Де це трапиться: пошук (Challenge 1): запит «Law & Order», вставлений у шаблонний рядок без кодування, шукає «Law ».

4.3. CORS

Опрацювати:

- Cross-Origin Resource Sharing (CORS), розділи Simple requests і Preflighted requests — MDN
- CORS-safelisted request header, Request: mode — MDN
- Документація TVmaze, розділ CORS

Дослідити: що таке origin; хто дозволяє доступ (сервер) і хто його забороняє (браузер); який запит «простий», а для якого браузер спершу надсилає `OPTIONS`; що означає `Access-Control-Allow-Origin: *`; що робить `mode: 'no-cors'`.

Експерименти:

11. **Два домени TVmaze.** `api.tvmaze.com` — API, `www.tvmaze.com` — сайт для людей.

```
const api = await fetch('https://api.tvmaze.com/shows/82');
console.log('api:', api.status);
try {
  const site = await fetch('https://www.tvmaze.com/shows/82');
  console.log('www:', site.status);
} catch (error) {
  console.log('www:', error.name, error.message);
}
```

    Проаналізуйте: що виведе кожен `console.log`? Чи дійшов запит до `www.tvmaze.com` і чи прийшла відповідь — що про це каже Network? Що саме заблокував браузер: надсилання запиту чи читання відповіді?
    Де це трапиться: AI-чернетки інколи беруть адресу сторінки сайту замість ендпоїнта API.

12. **Preflight.** Той самий запит до API двічі: із заголовком `Content-Type` і з `Accept`.

```
try {
  const response = await fetch('https://api.tvmaze.com/shows/82', {
headers: { 'Content-Type': 'application/json' } });
  console.log('Content-Type:', response.status);
} catch (error) {
  console.log('Content-Type:', error.name, error.message);
}
const response = await fetch('https://api.tvmaze.com/shows/82', { headers:
{ Accept: 'application/json' } });
console.log('Accept:', response.status);
```

    Проаналізуйте: що виведе кожен рядок? Скільки рядків з'явиться в Network для першого запиту, з яким методом і статусом? Документація TVmaze каже: «All endpoints are CORS enabled». Чому ж перший запит падає?
    Де це трапиться: пошук (Challenge 1). `headers: { 'Content-Type': 'application/json' }` — звичка з POST-запитів, яку AI-агенти часто переносять у GET.

13. **mode: 'no-cors'.** Той самий запит до сайту, що в експерименті 11, але з `mode: 'no-cors'`.

```
const response = await fetch('https://www.tvmaze.com/shows/82', { mode:
'no-cors' });
console.log(response.type, response.status, response.ok);
console.log(JSON.stringify(await response.text()));
```

    Проаналізуйте: чи буде помилка? Що виведуть `type`, `status`, `ok` і тіло відповіді?

Де це трапиться: «виправлення» CORS, яке пропонують AI-агенти: помилка зникає, але разом з нею зникають і дані.

4.4. Панель Network

Опрацювати: Inspect network activity, Network features reference (розділи Filter, Throttling, Block requests, Override content, Timing) — Chrome DevTools.

Дослідити на сторінці експериментів або будь-якому сайті:

- колонки Status, Type, Initiator, Size, Time і Waterfall; що означають `(canceled)`, `(failed)`, `(disk cache)` і `CORS error`;
- вкладки запиту: Headers (General, Response, Request), Payload, Preview, Response, Initiator, Timing (Waiting for server response — це TTFB);
- throttling (Slow 4G, Offline), **Request blocking** (Ctrl+Shift+P → «Show Request blocking»), Preserve log, Copy → Copy as fetch;
- **Override content**: як підмінити тіло відповіді, наприклад, HTML замість JSON.

Окремої таблиці для цього розділу немає: панель Network знадобиться в кожному challenge, а чек-лист Network у ТЗ (§6) — частина звіту.

4.5. Документація API → `API.md`

Опрацювати документацію TVmaze API: вступ (адреси, embedding), розділи Rate limiting, Caching, CORS, Licensing і ендпоїнти Show search, Show main information, Show episode list, Show cast.

Завдання. Складіть конспект `API.md` — таблицю, яку ви потім покладете в репозиторій лабораторної (§5.1) і з якої напишете правила для агента (§5.5). Кожен рядок перевірте справжнім запитом у Network і вкажіть адресу цього запиту.

Питання	Що каже документація	Що показав запит (адреса)
Ендпоїнт пошуку за назвою, параметри		
Форма відповіді пошуку		
Ендпоїнт серіалу; як отримати епізоди й акторів одним запитом		
Поля серіалу, що можуть бути `null` (з прикладом серіалу)		
Що повертає пошук без результатів		
Що повертає неіснуючий id		
Ліміт запитів і що робити на 429		

Питання	Що каже документація	Що показав запит (адреса)
Кешування: що кешує TVmaze і що — браузер		
CORS: що обіцяє документація і що показав експеримент 12		
Ліцензія й атрибуція		

4.6. Асинхронний код, написаний AI

Дослідити, що AI-агент зазвичай робить у коді запитів, якщо його про це не просили:

- не перевіряє `response.ok`, тож 404 і 500 стають «даними» або «нічого не знайдено»;
- додає `Content-Type: application/json` до GET-запиту;
- надсилає запит на кожне натискання клавіші або додає debounce, але не скасовує застарілі запити;
- вставляє запит у шаблонний рядок без кодування;
- запитує серіал, епізоди й акторів один за одним, не помічаючи `embed` у документації;
- лишає стан завантаження після помилки (немає `finally`);
- лише пише помилку в Console або показує `alert()`;
- показує скасований запит (`AbortError`) як помилку;
- на 429 повторює запит одразу й без кінця;
- читає заголовки, яких браузер не віддає скрипту для чужого домену;
- вигадує ендпоїнти й параметри, яких немає в документації;
- «виправляє» CORS через `mode: 'no-cors'` або публічний CORS-проксі.

Саме це шукайте в AI-чернетках у §5. Файл правил для агента (§5.5) — спосіб сказати агенту про це заздалегідь. Опрацюйте, як його читають агенти: Rules — Cursor, Adding repository custom instructions — GitHub Copilot.

5. Практична частина: AI-чернетка → аудит → виправлення

Starter-проєкт **ShowFinder** — пошук серіалів у TVmaze. У готовому застосунку користувач вводить назву, бачить список знайдених серіалів, вибирає серіал і отримує його сторінку: опис, епізоди за сезонами й акторів. Поки дані завантажуються, застосунок показує це; якщо щось пішло не так, пояснює, що саме, і пропонує спробувати ще. HTML, CSS, шаблони станів і модуль показу даних `src/view.js` уже готові. Порожні лише `src/api.js` (запити до TVmaze) і `src/app.js` (поведінка й стани).

Ваше завдання — довести застосунок до робочого стану за три кроки (challenges). Кожен challenge проходить той самий цикл (§5.2): AI-агент пише чернетку за коротким запитом, ви фіксуєте її в Git, перевіряєте й виправляєте власноруч. Виконуйте challenges по черзі: кожен спирається на попередні.

Як перевіряти. Точні правила — у ТЗ, §4. Вони пронумеровані: С3.2 — друге правило Challenge 3. Сторінка `check.html` перевіряє правила в браузері й показує їх блоками, по одному на challenge. На запити до TVmaze вона відповідає сама, записаними відповідями API: затримує їх, повертає 404, 429 чи 500, обриває з'єднання. Тому вона не залежить ні від мережі, ні від ліміту запитів. Те, чого `check.html` не бачить, — справжній TVmaze, справжню помилку CORS і панель Network — перевіряє чек-лист Network у ТЗ, §6.

5.1. Підготовка

1. Завантажте showfinder-starter.zip і розпакуйте в **окремий репозиторій** або в теку `lab-11/` спільного репозиторію з лабораторними. Зробіть перший коміт **без змін**.
2. Додайте `API.md` з §4.5 окремим комітом: `API notes`.
3. Відкрийте `index.html` через Live Preview у Chrome або Edge, як у §4: форма пошуку, порожні результати, «Виберіть серіал у результатах пошуку.», Console без помилок.
4. Відкрийте так само `check.html`: «Пройдено: 0 з 24». Прочитайте, чого перевірка не бачить.
5. `check.html`, `check/`, `offline.html` і `src/view.js` не змінюються в жодному коміті.

Ліміт TVmaze на занятті. Уся аудиторія виходить в інтернет з однієї IP-адреси, а TVmaze гарантує лише 20 запитів за 10 секунд з однієї адреси. Якщо Network показує відповіді 429, працюйте в `offline.html`, доки ліміт не відновиться: це той самий застосунок, але з записаними відповідями TVmaze. `check.html` справжнього API не використовує взагалі.

5.2. Правила роботи з AI-агентом

- **Дослідження (§4).** «Мій аналіз» пишете самі, без AI-агента й до запуску.
- **Challenges.** Кожен challenge виконуйте за чотири кроки. Приклади нижче — для Challenge 2.
    1. **Чернетка AI.** Перемкніть AI-агента в IDE в режим, у якому він сам змінює файли проєкту (зазвичай — Agent). Закрийте в редакторі вкладки з `check.html`, файлами `check/` і `API.md`: багато агентів додають відкриті файли до контексту. Вставте в чат **лише** запит із розділу цього challenge, слово в слово. ТЗ, `check.html` і власних уточнень агенту не давайте. Документацію TVmaze агент може відкрити сам — посилання є в запиті.

2. **Коміт чернетки.** Нічого не виправляючи, закомітьте все, що змінив агент, з повідомленням `Challenge 2: AI draft`. Комітьте, навіть якщо чернетка не працює. Єдиний виняток — файли, які не змінюються в жодному коміті (§5.1). Якщо агент їх змінив, перед комітом поверніть їх командою `git restore check.html check offline.html src/view.js` і запишіть це в підсумок звіту.
3. **Аудит.** Відкрийте `check.html` і знайдіть блок цього challenge. Для кожного ✗ з'ясуйте, яке правило ТЗ порушено і чому, і відтворіть це в `index.html` з доказом у Network: статус, заголовки, Payload, `(canceled)`, Initiator. Перегляньте й diff чернетки на обмеження ТЗ, §5.
4. **Виправлення власноруч.** AI може пояснити перевірку, правило, статус чи заголовок, але **не пише** код виправлення. Коли всі рядки блоку цього challenge і блоків попередніх позначені ✓, закомітьте виправлення одним комітом: `Challenge 2: fixes`.

Якщо чернетка з першого разу пройшла весь свій блок і не зламала попередніх, коміту виправлень немає, а в підсумку звіту вкажіть, як ви її перевірили. Агент часто робить більше, ніж просили, наприклад, уже в Challenge 1 обробляє помилки. Це теж частина чернетки: перевіряйте її в тому challenge, якому вона належить.

Приклад запиту для пояснення (режим без редагування — ask / plan):

> Поясни, чому падає ця перевірка. Нижче — її назва й повідомлення, запит із панелі Network і мій код. Не пиши виправлений код: назви механізм fetch, HTTP чи CORS, який я, ймовірно, не врахував, і як перевірити це в DevTools.

У звіті вкажіть назву та версію агента, модель і дату роботи.

5.3. Challenge 1 — пошук

Що має вийти. Користувач вводить назву, і застосунок шукає серіали в TVmaze, коли користувач перестає вводити, або одразу після Enter. Поки запит триває, область результатів показує «Завантаження…». Потім — список серіалів у порядку відповіді API і рядок статусу «Знайдено: N», а якщо нічого не знайдено — повідомлення про це. Відповідь на застарілий запит ніколи не замінює свіжішу.

Що вже є. Форма `#search` з полем `query`, рядок статусу `#status`, область `#results`, шаблони станів `#state-loading`, `#state-message` і `#state-error` в `index.html`. Функція `renderResults(container, shows)` у `src/view.js` показує масив серіалів картками.

Код: `searchShows()` у `src/api.js`, пошук і його стани в `src/app.js`.

Запит для AI-агента:

> Зроби пошук серіалів через TVmaze API (https://www.tvmaze.com/api): поле в формі #search, результати в #results через renderResults з src/view.js.

Перевірка: у блоці «C1 · Пошук» сторінки `check.html` всі рядки мають бути позначені ✓. Знадобляться експерименти 1, 6, 10 і 12.

5.4. Challenge 2 — сторінка серіалу

Що має вийти. Клік по картці показує праворуч (на вузькому екрані — нижче) сторінку серіалу: дані, опис, епізоди за сезонами й акторів. Поки сторінка завантажується, вона показує «Завантаження…», а список результатів лишається на місці й працює. Уже відкритий раніше серіал з'являється одразу, без нового запиту. Якщо вибрати інший серіал, поки попередній ще завантажується, показується лише останній вибраний.

Що вже є. Область `#details` і функція `renderShow(container, { show, episodes, cast })` у `src/view.js`. Що саме запитувати, щоб отримати епізоди й акторів, — у документації TVmaze і у вашому `API.md`.

Код: `getShow()` у `src/api.js`, вибір серіалу і його стани в `src/app.js`.

Запит для AI-агента:

> Коли користувач вибирає серіал у списку, покажи в #details його опис, епізоди й акторів через renderShow з src/view.js.

Перевірка: у блоці «C2 · Сторінка серіалу» і в блоці C1 всі рядки мають бути позначені ✓. Знадобляться експерименти 3, 5, 6 і 8.

5.5. Правила для агента: `AGENTS.md`

Після `Challenge 2: fixes` і перед Challenge 3 напишіть файл правил для агента. Цього разу агент отримає не лише короткий запит, а й контекст проєкту, і ви порівняєте його чернетку з попередніми.

- **Де.** `AGENTS.md` у корені відкритої в IDE теки: його читають Copilot, Cursor та інші агенти. Якщо ваш агент читає інший файл (наприклад, `.github/copilot-instructions.md`), покладіть текст туди й укажіть це у звіті.
- **Що.** До 40 рядків, своїми словами: контракт TVmaze з вашого `API.md` (ендпоїнти, форма відповідей, статуси, ліміт, CORS) і правила проєкту, які ви знайшли в аудиті C1–C2 (наприклад, що не так було з заголовками чи зі скасуванням). ТЗ цілком не копіюйте: у звіті треба показати, що агент узяв з контексту, а що ні.

- **Коміт:** окремий, `Agent rules`. Перед Challenge 3 переконайтеся, що агент бачить файл: більшість агентів показують підключені правила в контексті запиту.

5.6. Challenge 3 — помилки

Що має вийти. Кожен збій пояснено користувачеві в тій області, де він стався: «Серіал не знайдено», «Не вдалося отримати дані від TVmaze.» або «Немає з'єднання з TVmaze.», а поруч — «Спробувати ще», що повторює той самий запит. На 429 застосунок сам чекає кілька секунд і повторює запит один раз, як радить документація TVmaze. Стан завантаження завершується за будь-якого результату, невдалі відповіді не потрапляють у кеш, скасований запит не вважається помилкою, а технічні подробиці лишаються в Console.

Що вже є. Шаблон `#state-error` з кнопкою «Спробувати ще» і ваш `AGENTS.md`.

Код: `src/api.js` і `src/app.js`.

Запит для AI-агента:

> Додай обробку помилок і повторну спробу для всіх запитів.

Перевірка: у блоці «С3 · Помилки» і в блоках С1–С2 всі рядки мають бути позначені ✓. Відтворіть збої й у справжньому застосунку: Offline і Request blocking у Network, неіснуючий серіал. Знадобляться експерименти 1, 2, 4, 6 і 7.

5.7. Коміти

Історія Git лабораторної:

1. starter-проєкт без змін;
2. `API notes`;
3. `Challenge 1: AI draft`, `Challenge 1: fixes`;
4. `Challenge 2: AI draft`, `Challenge 2: fixes`;
5. `Agent rules`;
6. `Challenge 3: AI draft`, `Challenge 3: fixes`.

Якщо чернетка одразу пройшла свій блок і не зламала попередніх, коміту `fixes` немає (§5.2). На GitHub коміт `Challenge N: fixes` показує рівно те, що ви змінили в чернетці AI. На захисті ви пояснюєте за ним, що в чернетці було не так (§7).

Після Challenge 3 пройдіть чек-лист Network ТЗ на фінальній версії й опублікуйте проєкт на GitHub Pages або Vercel (інструкція — у `README.md` starter-проєкту).

6. Вимоги до звіту

Тема листа:

```
[Веб] ЛР-11: AI-агент для інтеграції REST API
```

У листі вкажіть репозиторій і адресу опублікованого проєкту — один із двох рядків, GitHub Pages або Vercel:

```
GitHub repository: https://github.com/<username>/<repository>
GitHub Pages: https://<username>.github.io/<repository>/
Vercel: https://<project>.vercel.app/
```

Звіт повинен містити:

1. таблицю експериментів §4 (усі 13);
2. посилання на `API.md` і `AGENTS.md` у репозиторії;
3. назву та версію AI-агента, модель, дату роботи;
4. результат `check.html` на опублікованій сторінці — «Пройдено: 24 з 24»;
5. чек-лист Network ТЗ (N1–N6) зі знімками екрана;
6. підсумок «контекст чи типова поведінка»: які правила С3 чернетка виконала завдяки `AGENTS.md`, які виконала б і без нього (їх виконували чернетки С1–С2), а які проігнорувала, хоча файл про них казав; що AI пропускав у чернетках С1–С2; якщо агент змінював файли, які змінювати не можна, — які саме.

7. Усний захист

Формат здачі передбачає усну співбесіду за результатами практичної реалізації. Виправлення пишете ви, тож на захисті змінюєте код наживо, без AI, і відповідаєте на питання з демонстрацією екрана.

Орієнтовні питання:

Аудит

- Покажіть одну знахідку в AI-чернетці: рядок `check.html`, що її виявив, доказ у Network і зміну в коміті `fixes`, яка її усуває.
- Змініть код наживо: блок «Сьогодні в ефірі» з ендпоїнта, який ви знайдете в документації TVmaze на захисті; «Спробувати ще» лише там, де повторна спроба може допомогти; мінімальна довжина запиту 3 символи — і яка перевірка тоді зміниться.

Fetch і Promise

- `fetch` повернув відповідь 404. У якому місці вашого коду вона стає помилкою?
- Що станеться, якщо в `try` написати `return fetch(url)` без `await`?

Скасування й стани

- Що показує Network для скасованого запиту і чому це не стан помилки?
- Чому відповідь на «gir» не може замінити відповідь на «girls» у вашому коді?

HTTP і CORS

- Чому `Content-Type` у GET-запиті ламає запит, хоча документація каже, що CORS увімкнено? Чи можна виправити помилку CORS з боку клієнта? Що робить `mode: 'no-cors'`?
- `embed` чи три запити: що кажуть розділи Caching і Rate limiting документації про кожен варіант?
- Повторний пошук показує в Network `(disk cache)`. Навіщо тоді кеш у застосунку?

AI

- Що змінив `AGENTS.md` у чернетці Challenge 3, а чого не змінив? Чому, на вашу думку?

8. Оцінювання

1. Успішний захист — **2 бали**.
2. Додатковий **1 бал** за однієї з умов:
   - термін здачі до **23.10.2026**, АБО
   - робота на парі.